INFORME DE LABORATORIO 11 · TECNOLOGÍA WEB I

Laboratorio 11 — SSH de nivel profesional: endurecimiento, SFTP enjaulado, fail2ban y túneles

Informe de resultados: sshd sin root ni contraseñas, SFTP enjaulado para comunicación, bloqueo de fuerza bruta con fail2ban y nftables, y túnel a la API interna

Asignatura	Tecnología Web I (SIS-0150) · Sexto semestre · Presencial
Master Class	11 de 15 — SSH endurecido y fail2ban
Modalidad	Individual
Criterio de verificación	C2 — Configura servicios de correo electrónico, TELNET y SSH (40 %).
Docente	M.Sc. Ing. Jimmy Nataniel Requena Llorentty
Estudiante	Lizeth Rodríguez · Fecha: 25 / 10 / 2026
Resultado	`verificar_lab11.sh lizeth` → **15 de 15 comprobaciones** ✅

1. Objetivo

Endurecer OpenSSH (sin root, solo claves, grupos permitidos, criptografía moderna y registro de huellas), crear un acceso SFTP enjaulado para el equipo de comunicación, bloquear la fuerza bruta con fail2ban y usar un túnel SSH hacia un servicio interno, hasta pasar el verificador de 15 comprobaciones.

2. Contexto del problema

El servidor del colegio saldrá a internet con su dominio público, y un servidor de prueba registró miles de intentos de acceso en un día. Además, la encargada de comunicación necesita subir las fotos de la feria de ciencias **sin** que nadie le dé la contraseña de administrador.

Diseño implementado

```
lizeth (sshadmins) ── clave ed25519 ──▶ sshd :22 ── shell + sudo
comunicacion (sftp-web) ── clave ──▶ sshd ── internal-sftp, jaula /srv/horizonte → /www
root / contraseñas / otros usuarios ──▶ sshd ── Permission denied (publickey)
bot 172.18.0.3 ── 4 fallas en 10 min ──▶ fail2ban ──▶ nftables reject (1 h, creciente)
PC Windows ── ssh -L 9081:127.0.0.1:8081 ═══▶ sshd ──▶ API eco 127.0.0.1:8081 (Lab 4)
```

3. Requisitos previos

Laboratorio 10 completo (`ssh horizonte` → `lizeth` con clave ed25519) sobre el contenedor `horizonte-servidor-lab02` (`srv-horizonte`, red Docker `alternativa_docker_default`, IP 172.18.0.2). Antes de tocar sshd se abrieron **dos sesiones** como root con `docker exec`: la ventana 1 para trabajar y la ventana 2 como salvavidas, que no se usó. Los archivos de `lab11` se copiaron con `docker cp` y se normalizaron con `sed -i 's/\r$//'`.

4. Archivos del laboratorio

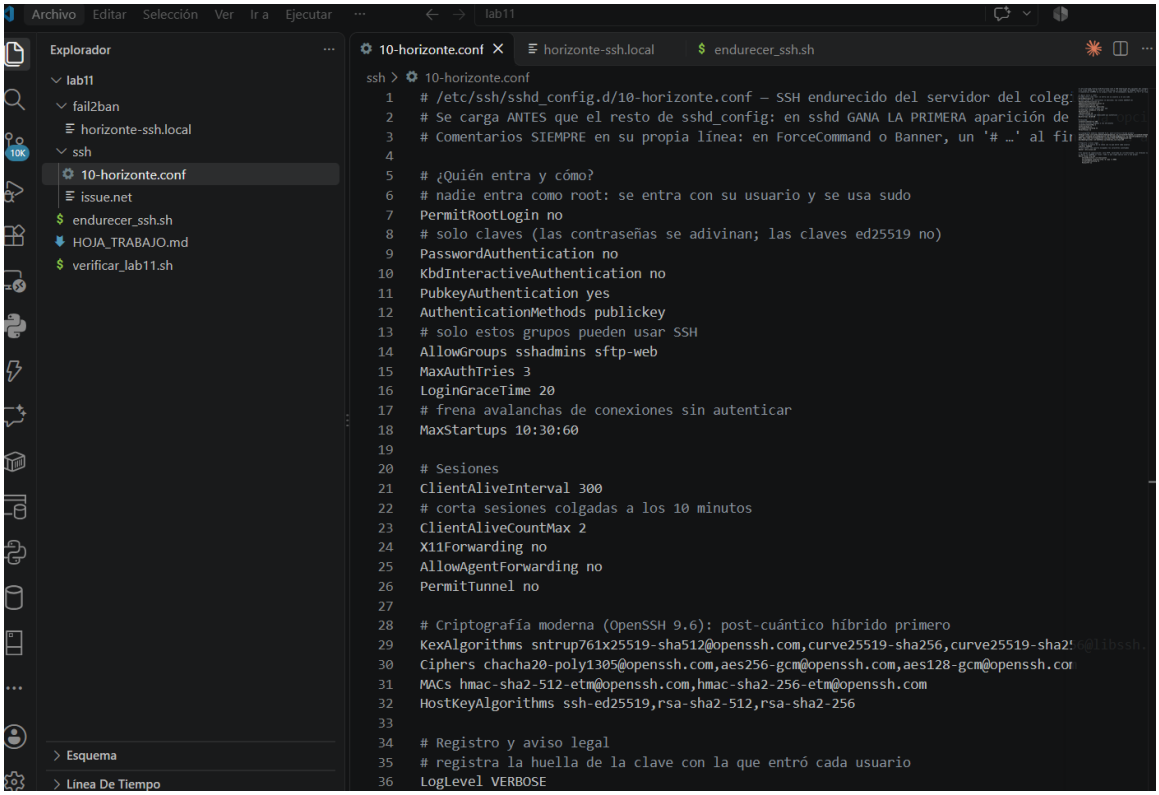
Tabla 1

Archivos del Laboratorio y TODO Completados

Archivo	TODO	Qué se completó
ssh/10-horizonte.conf	6	`PermitRootLogin no`, `PasswordAuthentication no`, `AllowGroups sshadmins sftp-web` y el bloque `Match Group sftp-web` (`ChrootDirectory`, `ForceCommand internal-sftp -d /www -u 0002`, `AllowTcpForwarding no`, `PermitTTY no`).
fail2ban/horizonte-ssh.local	2	`journalmatch = _SYSTEMD_UNIT=ssh.service + _COMM=sshd` · `maxretry = 4`, `findtime = 10m`, `bantime = 1h`.
endurecer_ssh.sh	1	`chown root:root /srv /srv/horizonte` y `chmod 755 /srv /srv/horizonte` (la jaula).
ssh/issue.net	—	Aviso legal en ASCII (completo).
verificar_lab11.sh	—	15 comprobaciones; resultado final **15 de 15**.

Figura 1

ssh/10-horizonte.conf con los TODO de root, contraseñas y AllowGroups



Nota. El bloque `Match Group sftp-web` está al final del archivo, fuera de esta vista; su contenido se comprobó con `grep` (Figura 6) y `sshd -T`.

Figura 2

fail2ban/horizonte-ssh.local con journalmatch, maxretry, findtime y bantime

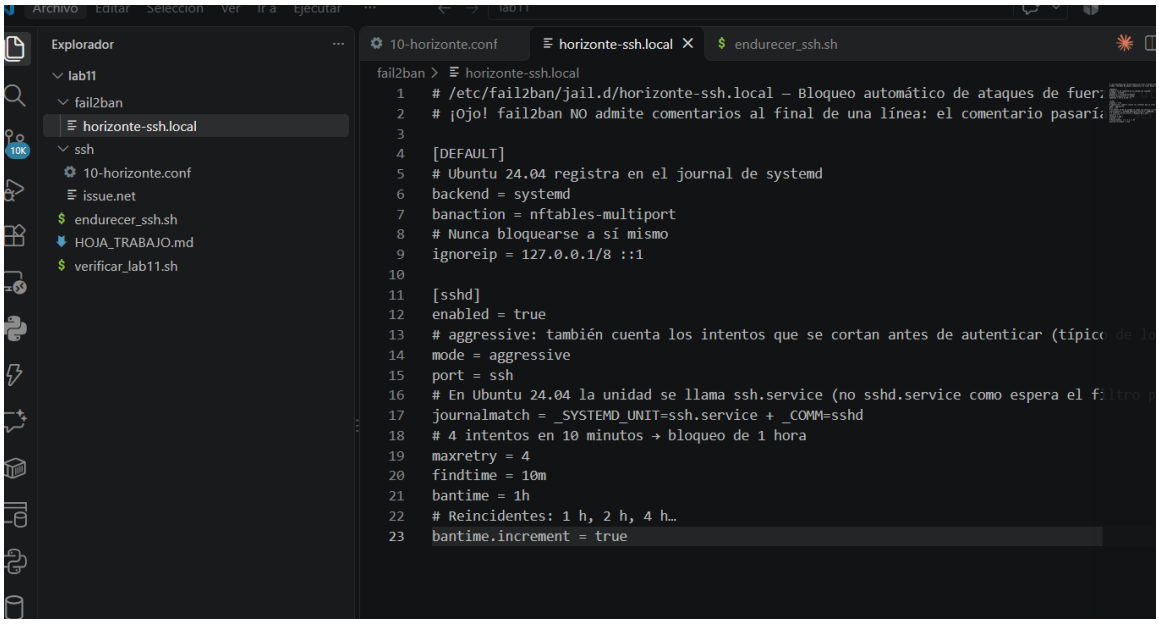
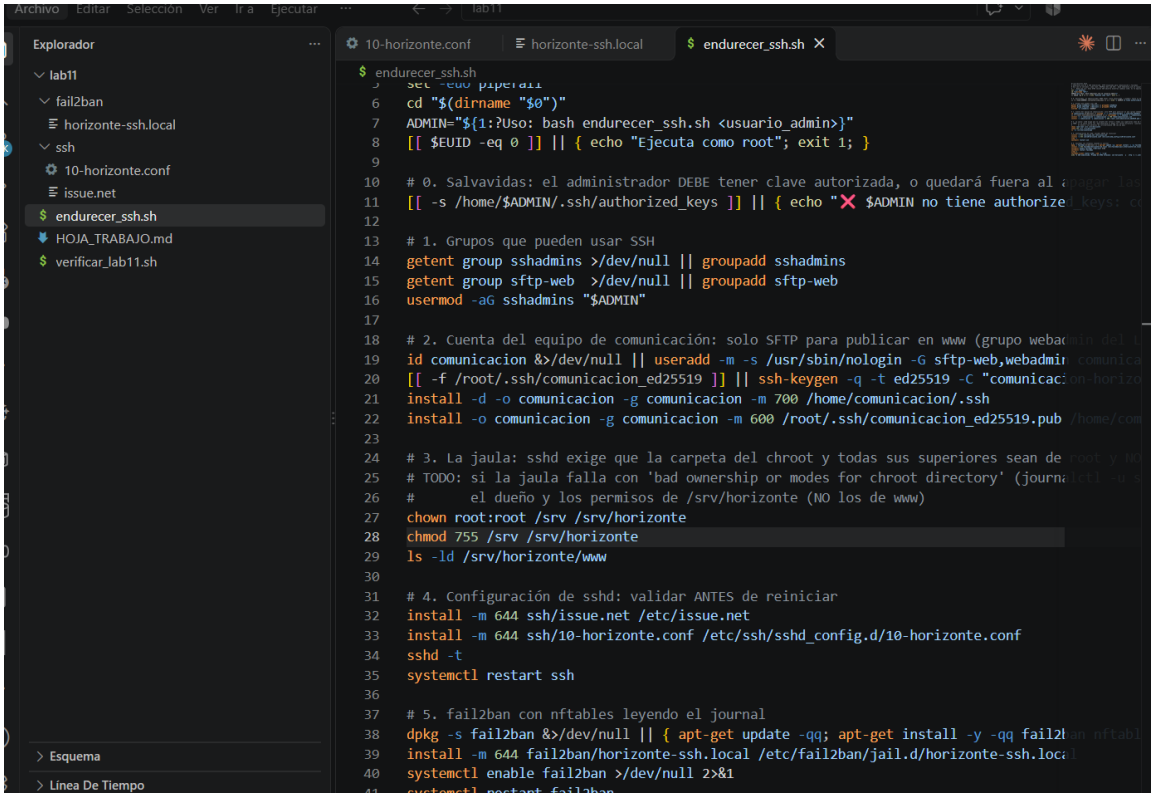


Figura 3

endurecer_ssh.sh: TODO de la jaula (líneas 27 y 28)



5. Desarrollo paso a paso

Paso 1 — Auditoría inicial

Parte A — cómo estaba sshd

`sshd -T` muestra la configuración **efectiva** (Figura 4). Root podía entrar con clave (`without-password`), se aceptaban contraseñas, X11 estaba activo, el registro era `INFO` y se aceptaban 10 algoritmos de intercambio de claves, incluidos los viejos. `journalctl` dio **0** intentos fallidos porque el servidor no está publicado. Uno expuesto recibe miles: en un *honeypot* sin proteger, Comparitech registró unos 73 000 ataques en 24 horas, **57 763** de ellos de fuerza bruta contra SSH (Security Magazine, 2021). La API de eco del Lab 4 escuchaba solo en `127.0.0.1:8081` (Figura 5).

Figura 4

Auditoría inicial con sshd -T

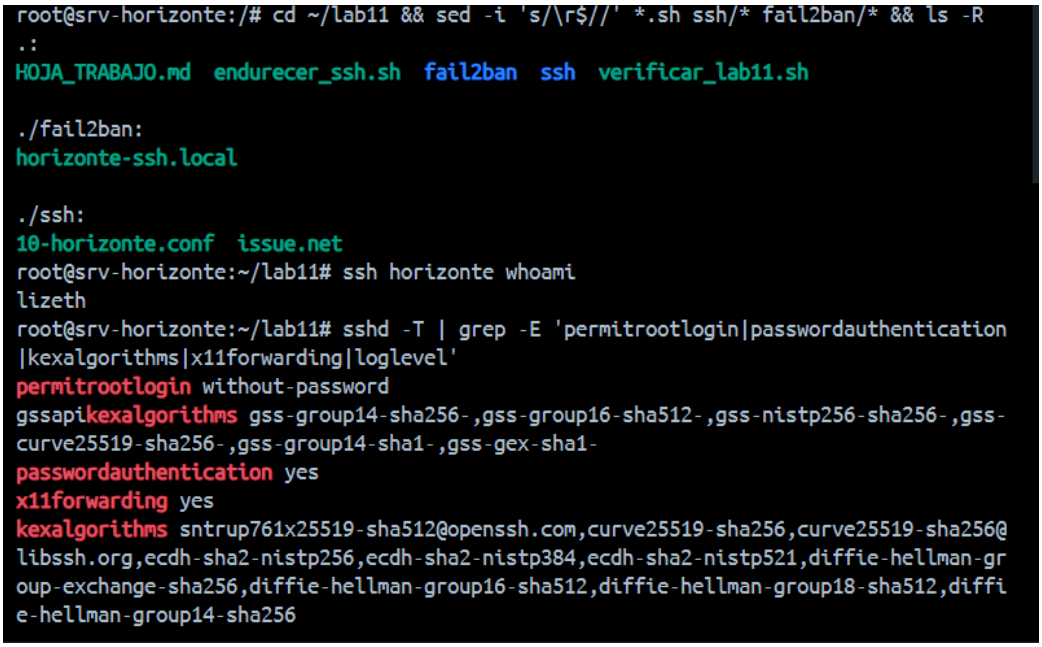


Figura 5

Cero intentos fallidos hoy y la API eco solo en 127.0.0.1:8081



Paso 2 — Endurecer sshd

Parte B — 10-horizonte.conf

Antes de aplicar se comprobaron los TODO con `grep` y el estado de las carpetas. `/srv/horizonte` era **`drwxrwsr-x root:webadmin`** (Figura 6), escribible por el grupo desde el Lab 2. `endurecer_ssh.sh lizeth` creó los grupos y la cuenta `comunicacion`, corrigió la jaula, instaló el aviso legal y la configuración, la **validó con `sshd -t` antes de reiniciar** e instaló fail2ban (Figura 7).

Figura 6

TODO revisados y carpetas antes de endurecer

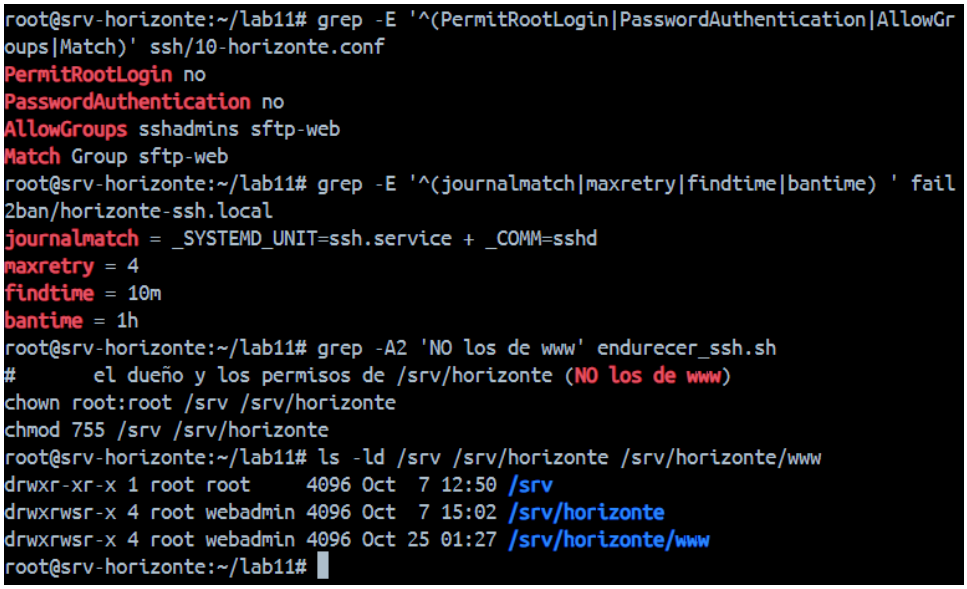
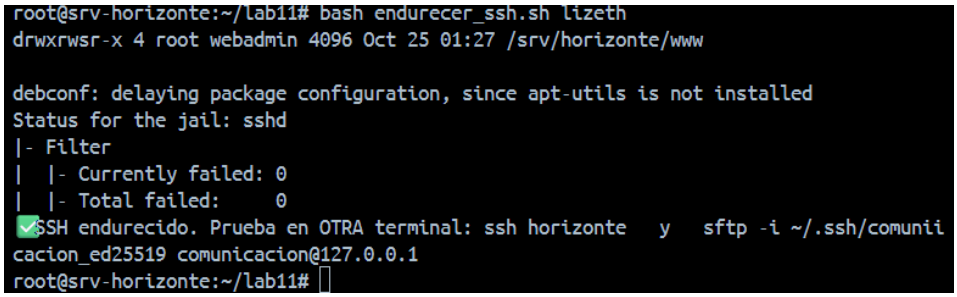


Figura 7

endurecer_ssh.sh aplicado sin errores



El archivo va en `/etc/ssh/sshd_config.d/` porque el `sshd_config` de Ubuntu empieza con `Include /etc/ssh/sshd_config.d/*.conf`. Se lee **antes** que el resto y en sshd **gana el primer valor leído**. Así se sobrescriben los valores del paquete sin tocar su archivo. Después del cambio, `sshd -T` muestra `permitrootlogin no`, `passwordauthentication no`, `x11forwarding no`, `loglevel VERBOSE`, `allowgroups sshadmins` y `sftp-web`, y solo **3** algoritmos de kex, con el post-cuántico primero (Figura 8).

Figura 8

Después de endurecer: la jaula es root:root y sshd -T con los valores nuevos

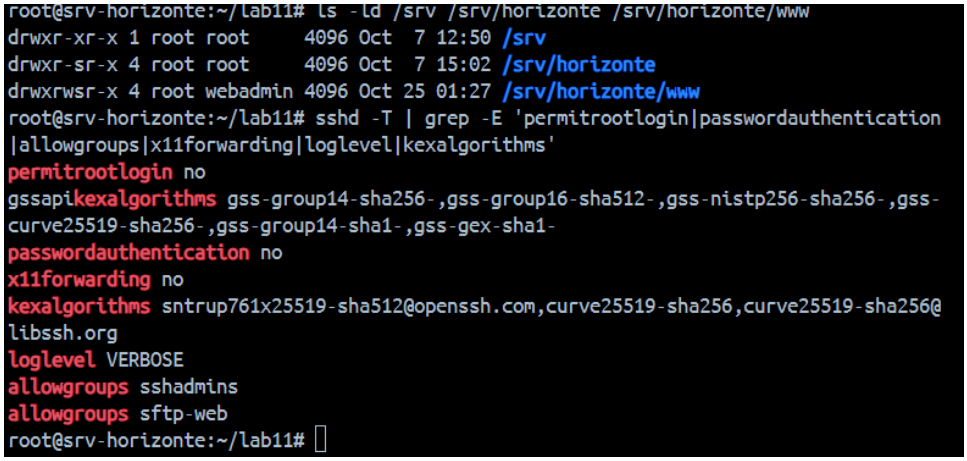


Tabla 2

Parte A y B: sshd Antes y Después

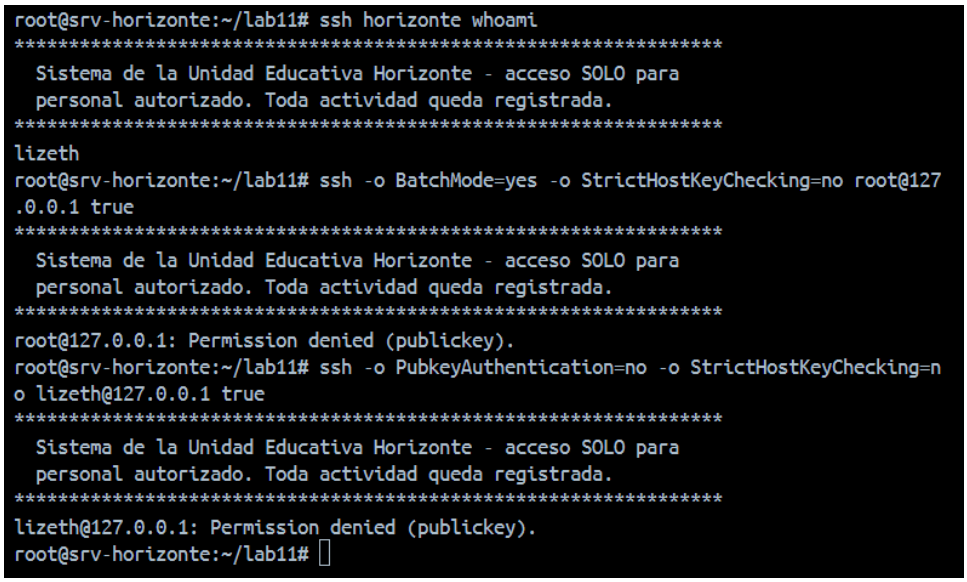
Opción	Antes	Después	Por qué
PermitRootLogin	without-password	no	Se entra con una cuenta personal y sudo: queda trazado quién hizo qué.
PasswordAuthentication	yes	no (+ AuthenticationMethods publickey)	Solo claves: no hay contraseña que adivinar ni robar.
AllowGroups	(todos)	sshadmins sftp-web	Solo esos dos grupos pueden siquiera intentarlo.
X11Forwarding	yes	no	Menos superficie de ataque.
LogLevel	INFO	VERBOSE	Registra la huella de la clave usada en cada acceso.

Opción	Antes	Después	Por qué
`KexAlgorithms`	10 algoritmos	`sntrup761x25519` + 2 curve25519	Solo intercambios modernos, el post-cuántico primero.

Las pruebas (Figura 9): `ssh horizonte` entra como **lizeth** y muestra el **aviso legal**. root recibe `Permission denied (publickey)` y lizeth **sin** clave también, porque el servidor ya no ofrece contraseña. El aviso está en **ASCII** a propósito: el cliente ssh muestra escapados los caracteres acentuados.

Figura 9

lizeth entra; root y el acceso sin clave son rechazados



> 🔎 **Verificado**
>
> root rechazado, contraseñas apagadas y `ssh horizonte` funcionando con clave.

Paso 3 — SFTP enjaulado

Parte C — el equipo de comunicación

`Bloque Match (al final del archivo)`

```
Match Group sftp-web
    ChrootDirectory /srv/horizonte
    ForceCommand internal-sftp -d /www -u 0002
    AllowTcpForwarding no
    PermitTTY no
```

`comunicacion` (grupos `sftp-web` y `webadmin`) entra con su propia clave y empieza en `/www`. Subió `aviso.txt`, que quedó **`-rw-rw-r-- comunicacion webadmin`**: el grupo viene del setgid de `www` y los permisos 664, de `-u 0002`. Al intentar `cd /etc` recibe `No such file`, porque para él `/` es `/srv/horizonte` (Figura 10). Dentro de la jaula `ls -l` muestra `1003 1002` y no nombres, porque ahí no existe `/etc/passwd`. Si pide una terminal, recibe **`This service allows sftp connections only.`** (Figura 11).

Figura 10

comunicacion publica por SFTP en /www y no puede salir de la jaula

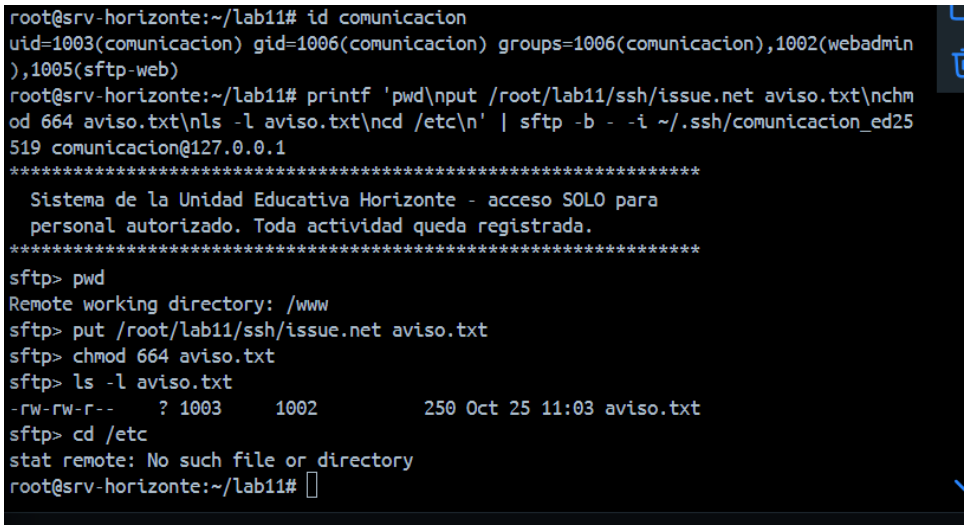
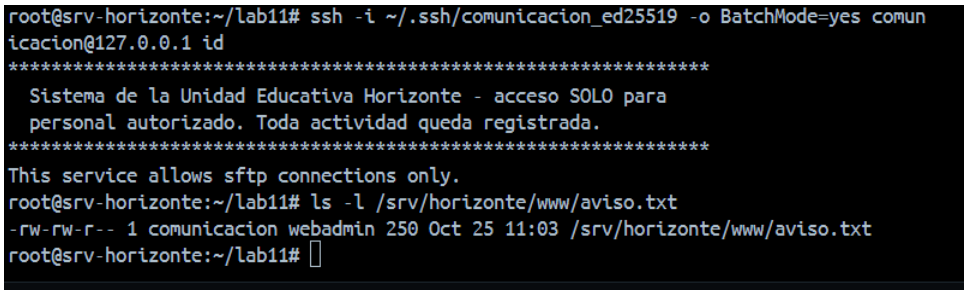


Figura 11

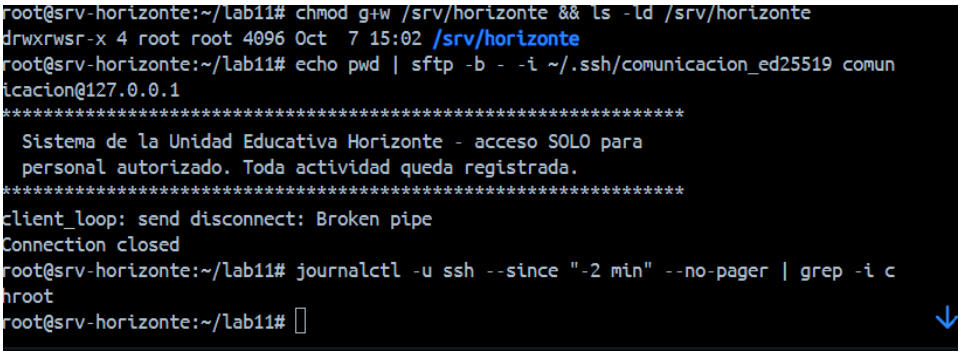
Sin terminal: solo SFTP; el archivo queda comunicacion:webadmin 664



La regla de la jaula. Para comprobarla, se devolvió un momento la escritura al grupo (`chmod g+w /srv/horizonte`): SFTP se cortó con `Connection closed` (Figura 12). Al quitarla volvió a funcionar (Figura 13). El motivo no apareció en `journalctl` (Figura 15), sino en `/var/log/auth.log`:

Figura 12

Con /srv/horizonte escribible por el grupo, sshd corta la sesión

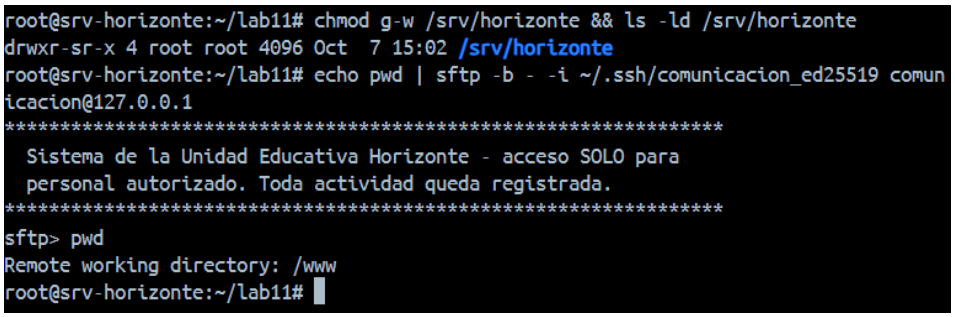


/var/log/auth.log (salida copiada de la terminal)

```
Oct 25 11:06:40 srv-horizonte sshd[2195]: fatal: bad ownership or modes for chroot directory
"/srv/horizonte"
Oct 25 11:07:23 srv-horizonte sshd[2208]: Changed root directory to "/srv/horizonte"
```

Figura 13

Permisos corregidos: la jaula vuelve a funcionar



sshd exige que la carpeta del `ChrootDirectory` **y todas sus superiores** (`/srv`, `/`) sean de **root** y **no** escribibles por el grupo ni por otros. Si el usuario enjaulado pudiera escribir en la raíz de su jaula, podría crear archivos que hagan las veces de `/etc` o `/lib` y escapar. Por eso la jaula es `root:root 755` y lo escribible es la subcarpeta `www` (`root:webadmin 2775`). El setgid que quedó en `/srv/horizonte` (`drwxr-sr-x`) no afecta, porque no da escritura.

> 🔎 **Verificado**
>
> `comunicacion` publica con 664 y grupo `webadmin`, no ve `/etc` y no obtiene terminal.

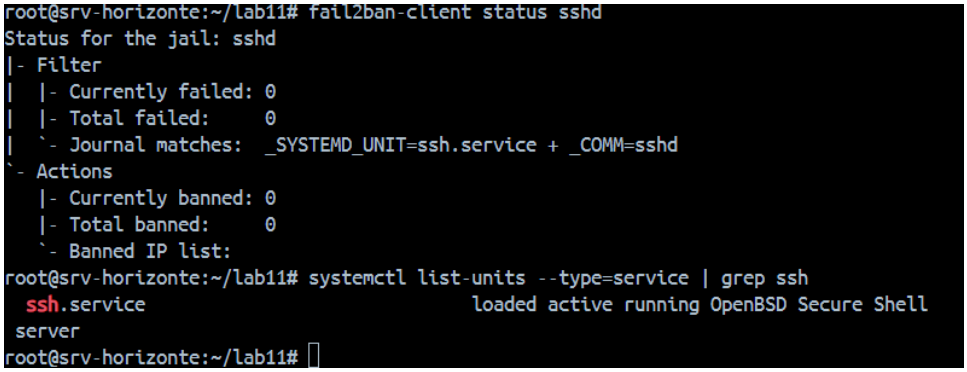
Paso 4 — fail2ban

Parte D — journalmatch y un problema que no estaba en la guía

`fail2ban-client status sshd` mostró `Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd`, y `systemctl list-units` confirmó que la unidad se llama **`ssh.service`** (Figura 14). El filtro por defecto busca `sshd.service`; sin el `journalmatch` corregido, fail2ban no vería nada. Hay otra trampa de **versión**: este servidor tiene OpenSSH 9.6, donde los mensajes los firma el proceso `sshd`. Desde OpenSSH 9.8 la autenticación la hace `sshd-session`, y entonces habría que ajustar `_COMM`.

Figura 14

fail2ban lee el journal de ssh.service



Pero `journalctl` **no tenía ningún mensaje** de sshd posterior al 24/10, aunque `auth.log` sí (Figura 15). El diagnóstico (Figuras 16 y 17) mostró la causa. `/dev/log` no era el enlace a journald sino el socket de **`/usr/sbin/syslogd`** (`inetutils-syslogd`). Según `dpkg.log`, ese paquete se instaló el **23/10 a las**

21:56:54 junto con `inetutils-telnetd`, al activar TELNET en el Lab 10. El script de ese lab purgó `telnetd`, pero dejó esas dos dependencias, y en el siguiente arranque `syslogd` tomó `/dev/log`. Sin journal, **fail2ban habría quedado ciego**.

Figura 15

El journal no muestra nada de comunicacion

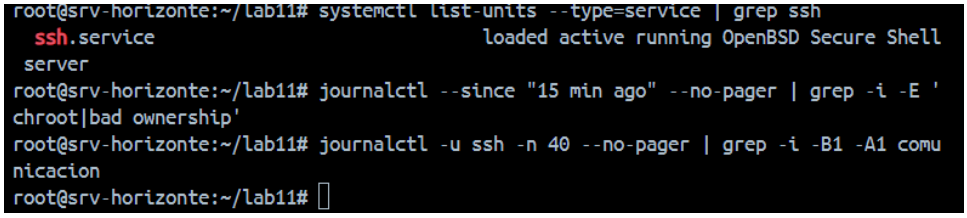


Figura 16

/dev/log pertenece a syslogd y no a journald

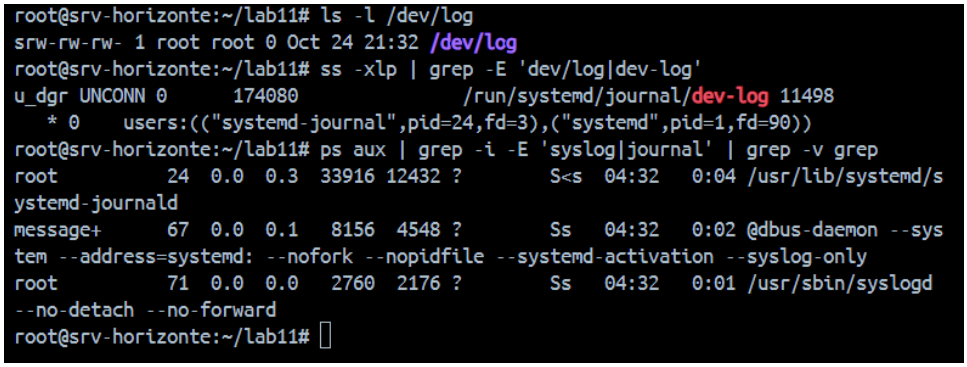
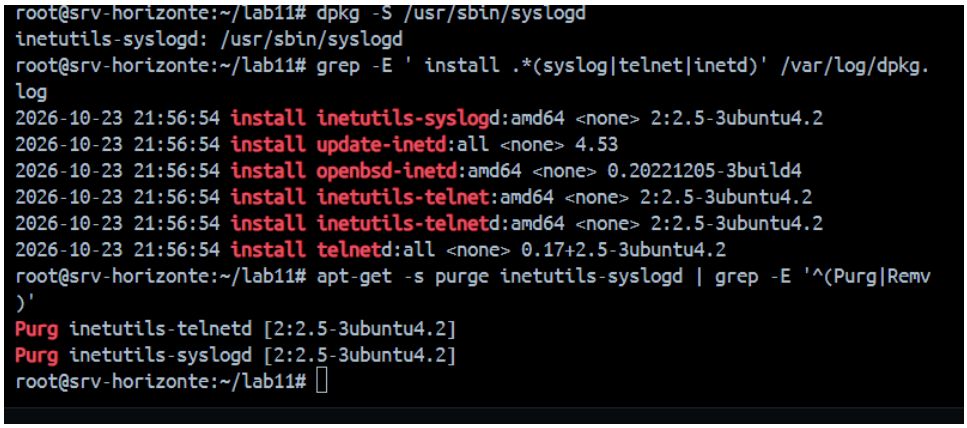


Figura 17

Origen: inetutils-syslogd, instalado con TELNET en el Lab 10; simulación de la purga



Primero se **simuló** la purga (`apt-get -s`), que mostró que también se iría `inetutils-telnetd`, el demonio de TELNET que había quedado. Luego se purgó, se restauró el enlace `/dev/log → /run/systemd/journal/dev-log` y se reiniciaron ssh y fail2ban. Desde entonces el journal registra la **huella de la clave** gracias a `LogLevel VERBOSE` (`Accepted publickey for lizeth … ED25519 SHA256:A36M7//…`, la clave del Lab 10), y el puerto 23 sigue cerrado (Figuras 18 y 19).

Figura 18

Purga de inetutils-syslogd (y del telnetd sobrante)

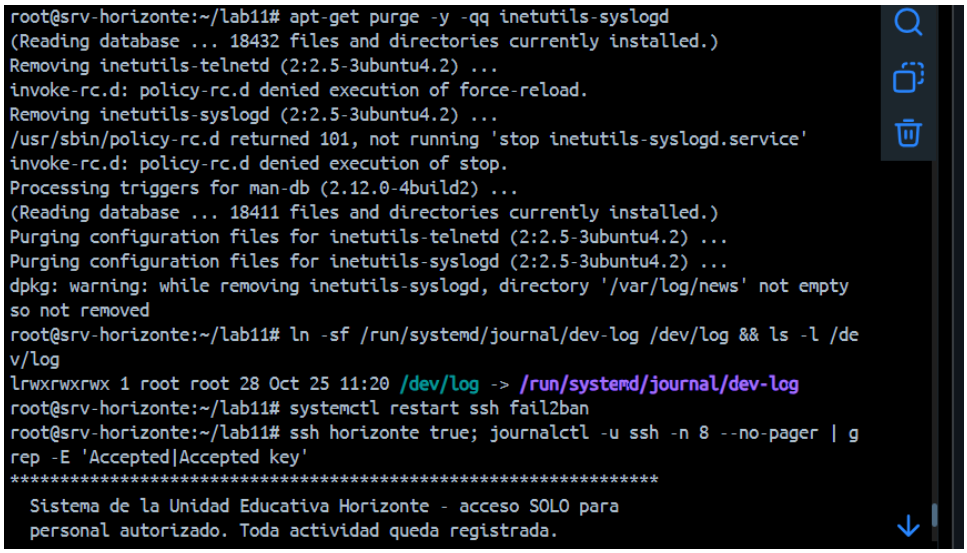
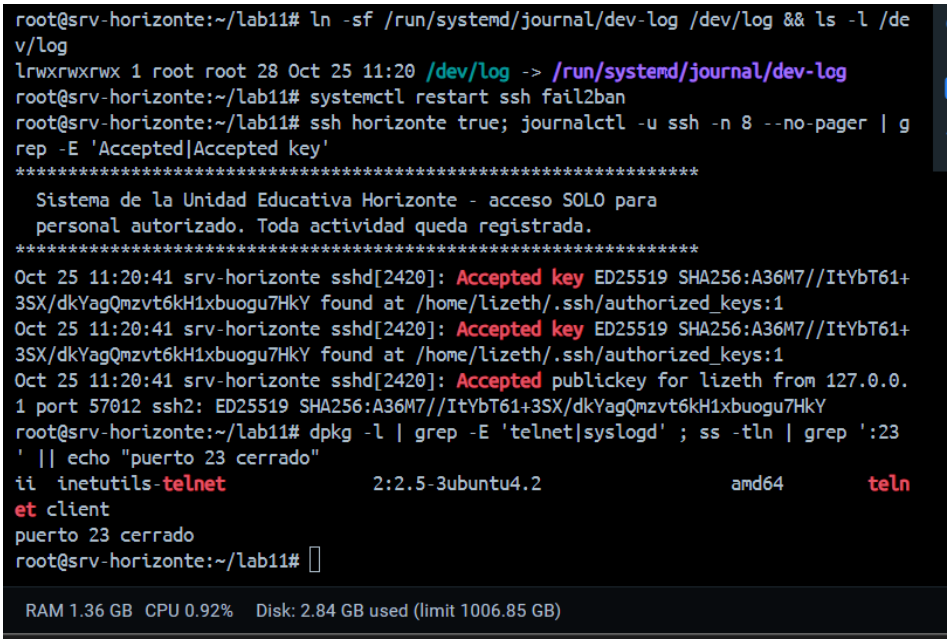


Figura 19

/dev/log restaurado: el journal registra la huella de la clave



Parte D — ataque simulado y bloqueo

El atacante fue un contenedor temporal, `tw1-cliente-fail2ban`, en la misma red Docker que el servidor (Figura 20). No podía ser el propio servidor, porque `ignoreip` excluye `127.0.0.1/8`. Con un usuario inexistente (`intruso`) intentó conectarse 6 veces a `172.18.0.2`. El primer ataque, a las **11:30:24**, quedó registrado en el journal (`Invalid user intruso from 172.18.0.3`) y en fail2ban: varios `Found 172.18.0.3` y **`Ban 172.18.0.3` a las 11:30:25.720**, en un segundo. La ejecución que muestra la Figura 21 fue posterior: la IP ya estaba bloqueada, por eso todos los intentos dieron `Connection refused`.

Figura 20

Red e IP del servidor (172.18.0.2)

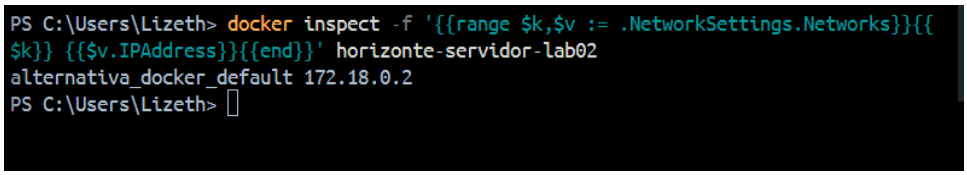
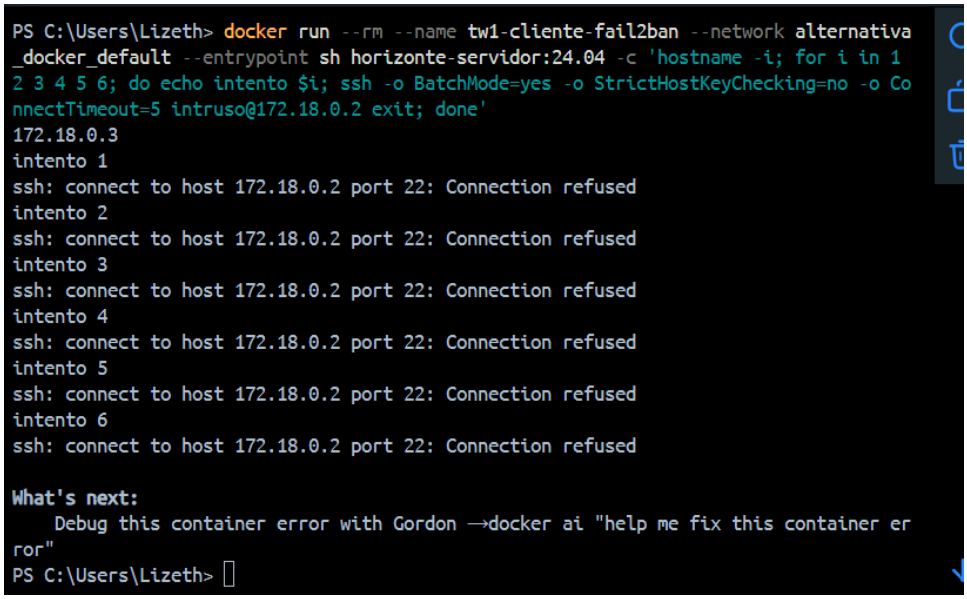


Figura 21

Atacante ya bloqueado: Connection refused desde el primer intento



`fail2ban-client` mostró `Banned IP list: 172.18.0.3` y nftables la tenía en el conjunto `addr-set-sshd`, con la regla `tcp dport 22 ip saddr @addr-set-sshd reject with icmp port-unreachable` (Figuras 22 y 23). El registro de fail2ban y el journal muestran la secuencia completa (Figuras 24 y 25). En modo `aggressive` cada conexión suma más de una falla (`Invalid user` y `Connection closed … [preauth]`). Las líneas `observer … # -> 2.0` corresponden a `bantime.increment`: a quien reincide se le duplica el bloqueo.

Figura 22

fail2ban: 172.18.0.3 bloqueada



Figura 23

nftables: tabla f2b-table, conjunto addr-set-sshd y regla reject

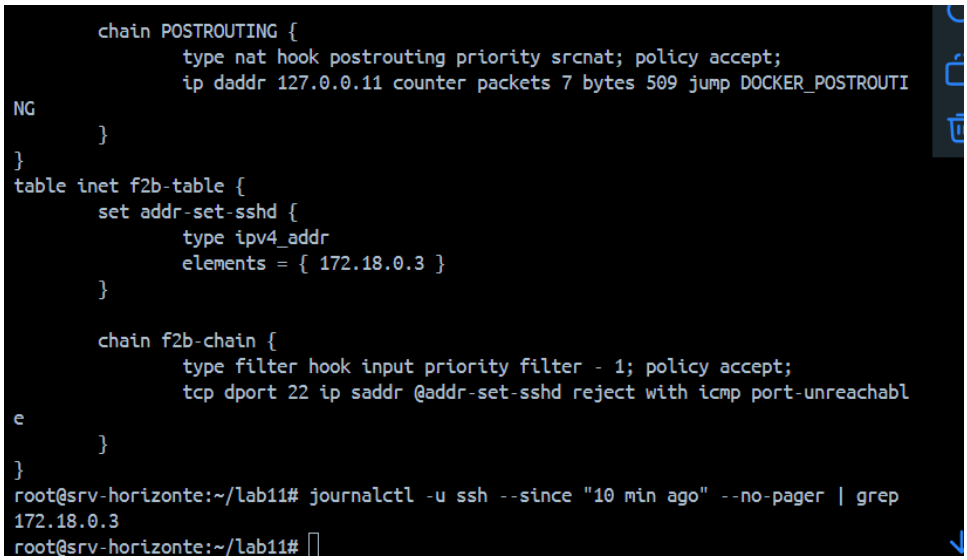


Figura 24

fail2ban.log: Found 172.18.0.3 en cada intento

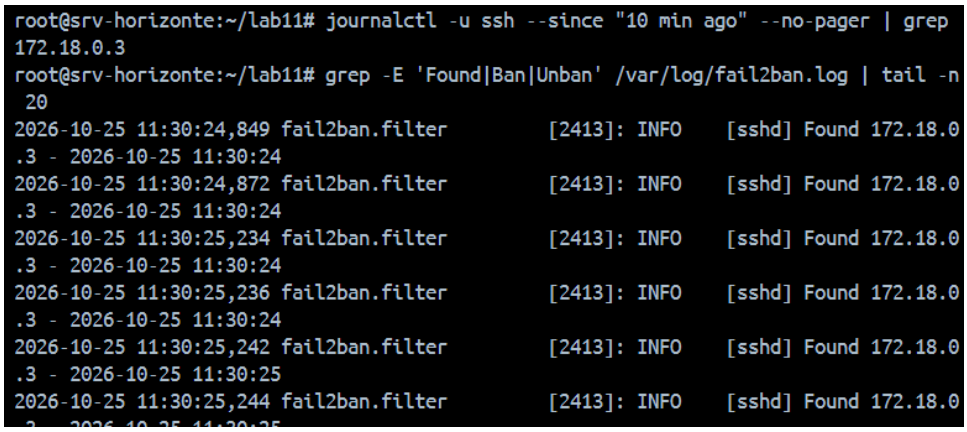
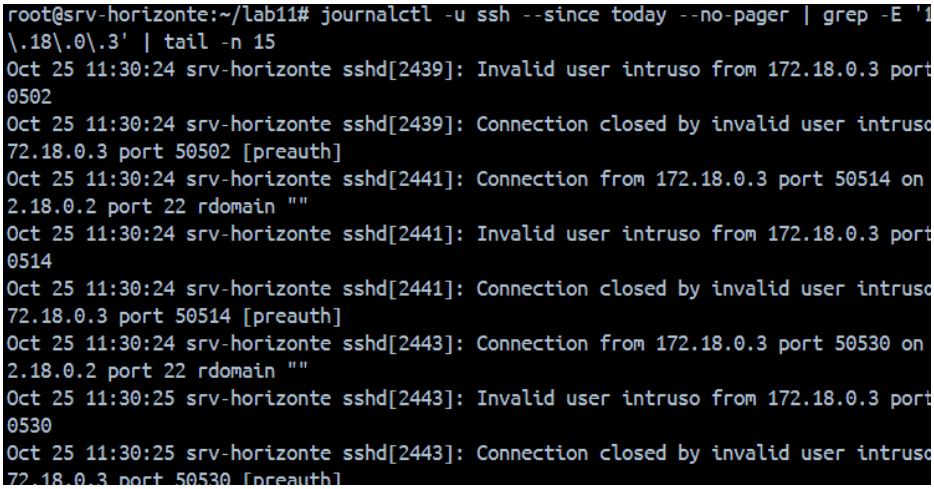


Figura 25

journal: Invalid user intruso from 172.18.0.3



Para registrar el **ciclo completo**, se desbloqueó la IP con `fail2ban-client set sshd unbanip` y el conjunto quedó vacío (Figura 26). Luego se repitió el ataque (Figura 27): los intentos **1 a 3** dieron `Permission denied (publickey)` y, ya bloqueada, los **4 a 6** dieron `Connection refused`. El estado final fue `Total failed: 20`, `Total banned: 2` y la secuencia `Ban 11:30:25 → Unban 11:46:10 → Ban 11:47:03` (Figura 28). Al terminar se desbloqueó para dejar el servidor limpio (Figura 29).

Figura 26

Desbloqueo manual: conjunto vacío

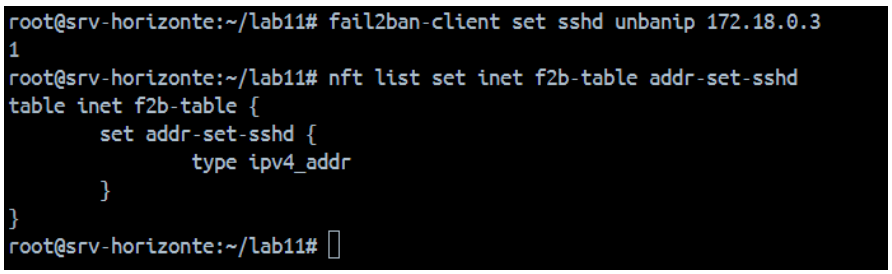
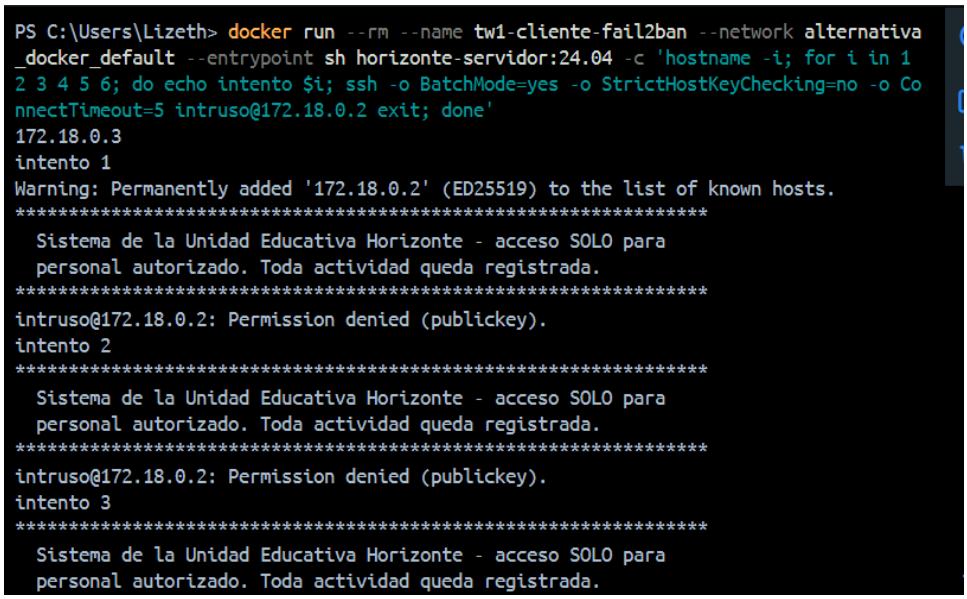


Figura 27

Segundo ataque: Permission denied y luego bloqueo



Nota. La captura recorta la salida en el tercer intento; en la misma ejecución, los intentos 4 a 6 respondieron `Connection refused` (Tabla 3).

Figura 28

Estado tras el segundo ataque: Ban, Unban y Ban en el registro



Figura 29

Desbloqueo final

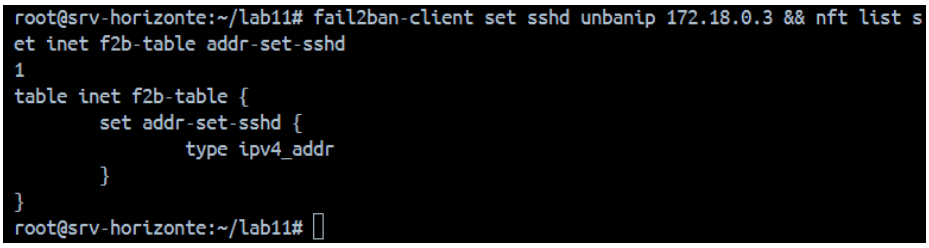


Tabla 3

Parte D: Respuesta del Servidor Durante el Ataque

Intento	Lo que ve el atacante	Lo que pasa en el servidor
1 a 3	`Permission denied (publickey)`	sshd: `Invalid user intruso from 172.18.0.3`; fail2ban: `Found` por cada línea.
4 a 6	`Connection refused`	fail2ban: `Ban 172.18.0.3`; nftables: `reject` antes de llegar a sshd.

> 🔎 **Verificado**
>
> `Journal matches: _SYSTEMD_UNIT=ssh.service` y la IP del atacante dentro de `addr-set-sshd`.

Paso 5 — Túnel

Parte E — la API interna desde la PC

El túnel se hizo desde **la PC con Windows** (OpenSSH_for_Windows 9.5p2), con una clave propia `lizeth-pc-2026` protegida con frase. Solo la `.pub` se copió a `authorized_keys`. En la primera conexión apareció **`REMOTE HOST IDENTIFICATION HAS CHANGED`**: la PC conservaba la clave de la imagen, anterior a regenerarla en el Lab 10. Como dice la reflexión de ese laboratorio, primero se verificó la huella **por otro canal** (`docker exec … ssh-keygen -lf` → `SHA256:KueRdm7E…`, igual a la del aviso). Solo entonces se borró esa entrada con `ssh-keygen -R "[localhost]:2222"` (Figuras 30 y 31).

Figura 30

Huella del servidor verificada por otro canal

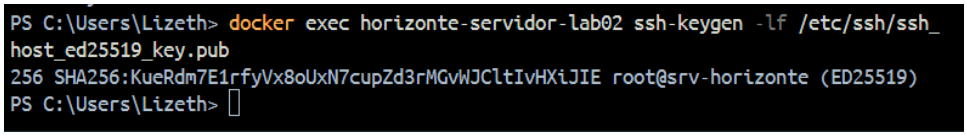
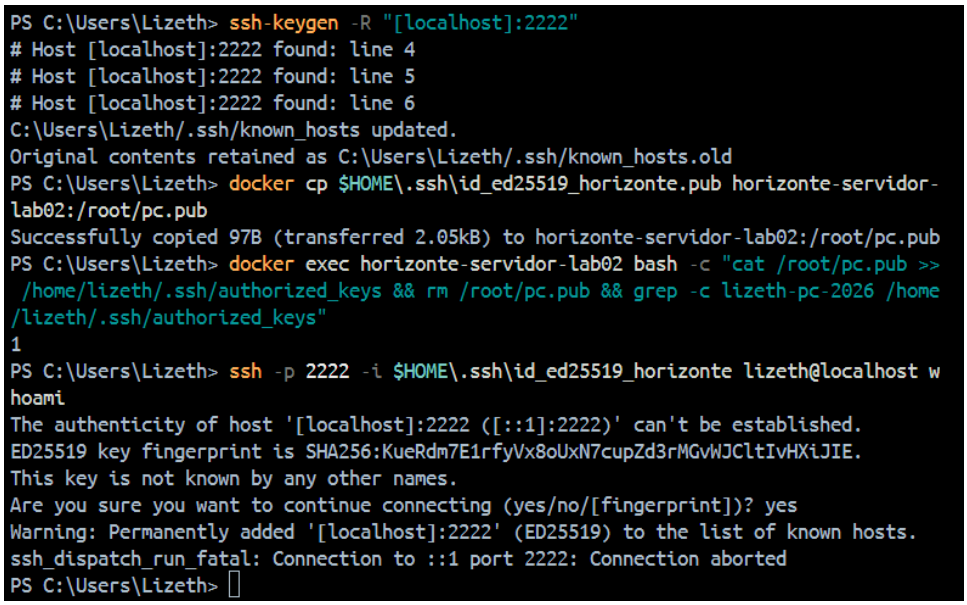


Figura 31

Entrada vieja borrada, clave de la PC autorizada y huella aceptada



Las dos primeras conexiones se cortaron (`Connection aborted`). `ssh -v` mostró que el intercambio de claves terminaba bien y el servidor cerraba justo antes de la autenticación (Figura 32). La causa fue **`LoginGraceTime 20`**: la pregunta de la huella quedó abierta más de 20 segundos y sshd cortó la conexión. Es la propia configuración endurecida en acción. Con la huella ya guardada, la PC entró como **lizeth** (Figura 33).

Figura 32

ssh -v desde Windows: el corte llega antes de autenticar

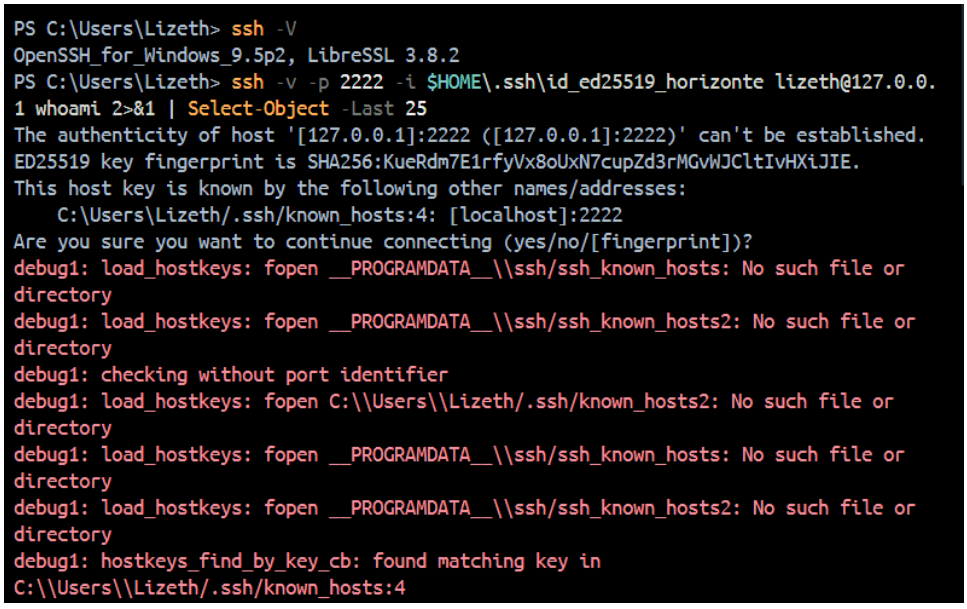
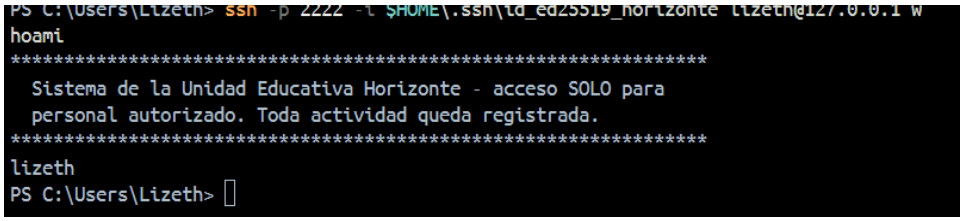


Figura 33

Acceso desde la PC con su propia clave



Túnel local (PowerShell)

```
ssh -N -L 9081:127.0.0.1:8081 -p 2222 -i $HOME\.ssh\id_ed25519_horizonte lizeth@127.0.0.1
curl.exe http://127.0.0.1:9081/eco
```

Con el túnel abierto, `curl` a un puerto **de la PC** devolvió el JSON de la API interna (Figura 34). La respuesta dice `"cliente": "127.0.0.1"`: para la API, la petición la hizo sshd **dentro** del servidor. La API sigue cerrada al mundo y solo se alcanza con una clave autorizada.

Camino del paquete

```
PC: curl ──► 127.0.0.1:9081 (ssh escucha en la PC)
        ╟ túnel cifrado por 2222 → 22
        ▼
srv-horizonte: sshd ──► 127.0.0.1:8081 (API eco) ──► respuesta por el mismo túnel
```

Figura 34

La API interna alcanzada desde la PC por el túnel SSH



`-D 1080` abre un túnel **dinámico**: un proxy SOCKS en la PC cuyo tráfico sale desde el servidor hacia cualquier destino, por ejemplo para navegar como si se estuviera en la red del colegio. **`-J`** conecta **saltando** por un bastión (`ssh -J lizeth@bastion lizeth@interno`): solo el bastión se expone y la sesión va cifrada de extremo a extremo.

> 🔎 **Verificado**
>
> `curl http://127.0.0.1:9081/eco` responde a través del túnel.

Paso 6 — Verificación

El verificador aprobó las **15 comprobaciones** (Figura 35). Entre ellas, que el journal guarda la huella de la clave, algo que solo fue posible después de corregir `/dev/log`.

Figura 35

verificar_lab11.sh: 15 de 15 comprobaciones



Reflexión — ¿claves y AllowGroups o cambiar el puerto?

Cambiar el puerto 22 es **seguridad por oscuridad**: reduce el ruido de los bots más simples, pero un escaneo encuentra el puerto nuevo en segundos y el servidor seguiría aceptando contraseñas adivinables. Las claves eliminan el riesgo de raíz (no hay contraseña que adivinar ni robar), `AllowGroups` limita quién puede siquiera intentarlo, `PermitRootLogin no` obliga a usar cuentas personales trazables y fail2ban corta a quien insiste. Mover el puerto puede sumarse como capa extra, pero nunca reemplaza estas medidas.

> 🔎 **Verificado**
>
> **Resultado: 15 de 15 comprobaciones.**

6. Retos (para llegar a Estratégico)

Los retos opcionales **no se realizaron** en esta entrega. Se dejan indicados con lo que haría falta para cada uno.

Tabla 4

Estado de los Retos

Reto	Estado	Qué haría falta
Analizar el servidor con `ssh-audit` y comentar cada advertencia	No realizado	`pip install ssh-audit` y `ssh-audit 127.0.0.1`; contrastar sus avisos con `HostKeyAlgorithms`, `Ciphers` y `MACs` de `10-horizonte.conf`.
Jaula de fail2ban para el portal Nginx (accesos fallidos del Lab 4)	No realizado	Un filtro con la expresión del registro de acceso fallido y una jaula `[nginx-horizonte]` con `logpath` al `access.log` del sitio.
Bastión: llegar a ns1 de red_dns saltando por el servidor (`ssh -J`)	No realizado	Conectar el servidor a la red de red_dns e instalar sshd en ns1; luego `ssh -J lizeth@srv-horizonte usuario@10.10.10.2`.

7. Uso de Inteligencia Artificial en este laboratorio

✅ Permitido — cómo se usó

- Explicación de cada opción de `sshd_config` y de fail2ban, comprobada con `sshd -T` y `fail2ban-client`.
- Interpretación de `journalctl`, `auth.log`, `fail2ban.log` y `ssh -v`; diagnóstico del `syslogd` que robaba `/dev/log`.
- Comparación con buenas prácticas de endurecimiento (claves, `AllowGroups`, kex moderno, `LogLevel VERBOSE`).

⛔ No permitido — cómo se respetó

- No se generaron diccionarios de contraseñas: el ataque fueron 6 intentos de un usuario inexistente, solo contra el servidor del curso.
- No se pegaron claves privadas: las de `comunicacion` y de la PC nunca se mostraron. Los registros solo tienen IPs del laboratorio.
- No se desactivó ninguna protección: la jaula se rompió a propósito un momento y se restauró; la huella se verificó antes de aceptarla.

La bitácora completa, con las 12 consultas realizadas, cómo se verificó cada una y qué se aprendió, está en el **Anexo B**.

8. Entregable

Tabla 5

Lista de Comprobación del Entregable

Elemento	Estado	Evidencia
`ssh/`, `fail2ban/` y `endurecer_ssh.sh` completos	✅	9 de 9 TODO (Tabla 1, Figuras 1 a 3)
Captura del verificador con 15 de 15	✅	Figura 35
Captura del ban en fail2ban	✅	Figuras 22, 23 y 28
`HOJA_TRABAJO.md` (partes A a F)	✅	Anexo A
`BITACORA_IA.md`	✅	Anexo B
Actividad 3: verificadores de los labs 10 y 11	✅	Lab 10: 11/11 · Lab 11: 15/15

9. Rúbrica de evaluación: evidencias por criterio

Tabla 6

Criterios de la Rúbrica y Dónde se Evidencian

Criterio	Puntos	Evidencia en este informe
sshd endurecido	25	Sin root, solo claves, `AllowGroups`, kex moderno y `VERBOSE` (Tabla 2, Figuras 8 y 9).
SFTP enjaulado	25	Jaula funcionando y regla del chroot demostrada con el error real (Figuras 10 a 13).
fail2ban	20	`journalmatch` correcto, problema de `/dev/log` resuelto, ban demostrado en nftables (Figuras 14 a 29, Tabla 3).
Túneles	15	Túnel local desde la PC a la API interna; `-D` y `-J` explicados (Figuras 30 a 34).
Trampas y bitácora	15	Comentarios al final de línea, banner ASCII, versión de OpenSSH (`_COMM`), `LoginGraceTime`; bitácora verificada (Anexo B).

10. Conclusiones

El servidor del colegio quedó listo para salir a internet. Solo entran por SSH los grupos autorizados y siempre con clave: root y las contraseñas quedaron fuera. Se negocia primero el intercambio post-cuántico, y cada acceso queda registrado con la huella de la clave usada. La encargada de comunicación publica en `/www` con su propia cuenta, encerrada en la jaula y sin terminal, sin conocer ninguna contraseña de administrador. Quien insiste con intentos fallidos queda bloqueado en nftables en menos de dos segundos.

El hallazgo más valioso no estaba en la guía. Un `syslogd` que llegó como dependencia de TELNET en el Lab 10 se apropió de `/dev/log`, y sin el journal fail2ban no habría visto ningún ataque. Esto muestra dos cosas: desinstalar algo no siempre limpia lo que trajo consigo, y una protección hay que **probarla** (aquí, con un ataque real y controlado) en lugar de suponer que funciona.

Referencias

Fail2ban. (s.f.). *Fail2ban* [Software y documentación]. GitHub. https://github.com/fail2ban/fail2ban

OpenBSD Project. (s.f.). *sshd_config(5) — OpenSSH daemon configuration file*. OpenBSD manual pages. https://man.openbsd.org/sshd_config

OpenBSD Project. (2024). *OpenSSH 9.8 release notes*. OpenSSH. https://www.openssh.com/txt/release-9.8

Requena Llorentty, J. N. (2026). *Guía de laboratorio 11: SSH de nivel profesional: endurecimiento, SFTP enjaulado, fail2ban y túneles* [Guía de laboratorio]. Universidad Privada Domingo Savio.

Security Magazine. (2021, 26 de abril). *Unsecured computer attacked 51 times per minute*. https://www.securitymagazine.com/articles/95082-unsecured-computer-attacked-51-times-per-minute

Ylonen, T., & Lonvick, C. (Ed.). (2006). *The Secure Shell (SSH) connection protocol* (RFC 4254). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc4254

Anexo A — HOJA_TRABAJO.md

Laboratorio 11 — Hoja de trabajo: SSH endurecido, SFTP enjaulado, fail2ban y túneles.

Nombre: Lizeth Rodríguez Fecha: 25/10/2026

Requisito cumplido: Laboratorio 10 completo (`ssh horizonte` → `lizeth` con clave ed25519, verificador 11/11). Servidor: contenedor `horizonte-servidor-lab02` (`srv-horizonte`, OpenSSH 9.6p1, red Docker `alternativa_docker_default`, IP 172.18.0.2). Trabajé como root con **dos sesiones abiertas** (`docker exec`): la ventana 1 para trabajar y la ventana 2 como salvavidas, sin tocarla.

Parte A — Auditoría inicial

1. `sshd -T` **antes** de endurecer:

Opción	Valor inicial	Riesgo
`permitrootlogin`	`without-password`	root podía entrar por SSH con clave
`passwordauthentication`	`yes`	acepta contraseñas, que se pueden adivinar por fuerza bruta
`x11forwarding`	`yes`	reenvío gráfico innecesario en un servidor
`kexalgorithms`	`sntrup761x25519-sha512@openssh.com,…,diffie-hellman-group14-sha256`	post-cuántico primero, pero acepta 10 algoritmos, incluidos los viejos
`loglevel`	`INFO`	no registra con qué clave entró cada usuario

2. `journalctl -u ssh --since today | grep -c 'Failed\|Invalid'` → **0**: el servidor no está publicado en internet. Uno expuesto recibe muchísimos intentos. En un estudio con *honeypots* sin proteger, Comparitech registró unos **73 000 ataques en 24 horas** (≈ 51 por minuto), y **57 763** de ellos fueron intentos de fuerza bruta contra SSH (Security Magazine, 2021). Un servidor del colegio con el puerto 22 abierto recibiría desde cientos hasta miles de intentos por día, de bots que prueban `root`, `admin` y contraseñas comunes.

Parte B — Endurecer sshd (6 TODO en `ssh/10-horizonte.conf`)

```
PermitRootLogin no
PasswordAuthentication no
AllowGroups sshadmins sftp-web
Match Group sftp-web
    ChrootDirectory /srv/horizonte
    ForceCommand internal-sftp -d /www -u 0002
    AllowTcpForwarding no
    PermitTTY no
```

`sshd -T` **después**: `permitrootlogin no`, `passwordauthentication no`, `x11forwarding no`, `loglevel VERBOSE`, `allowgroups sshadmins` / `allowgroups sftp-web` y `kexalgorithms sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha256@libssh.org` (solo 3, el post-cuántico primero).

Pruebas:

- `ssh horizonte whoami` → banner legal y `lizeth`.
- `ssh root@127.0.0.1` → `Permission denied (publickey)`.

- `ssh -o PubkeyAuthentication=no lizeth@127.0.0.1` → `Permission denied (publickey)`: el servidor ya ni ofrece contraseña.

1. ¿Por qué el archivo va en `sshd_config.d/` y por qué ahí "gana la primera aparición"? En Ubuntu 24.04, `/etc/ssh/sshd_config` empieza con `Include /etc/ssh/sshd_config.d/*.conf`, así que nuestro archivo se lee **antes** que el resto. En sshd, para casi todas las opciones **vale el primer valor que se lee** y los siguientes se ignoran. Por eso nuestro archivo manda sobre los valores de Ubuntu sin editar el `sshd_config` original, que puede reemplazarse en una actualización. Cada cambio queda en un archivo propio, fácil de revisar y de versionar.

2. `AuthenticationMethods publickey` exige que la autenticación sea **solo** por clave pública: aunque otra opción quedara activada por error, no se acepta ningún otro método. **`LogLevel VERBOSE`** registra la **huella de la clave** con la que entró cada usuario, por ejemplo `Accepted publickey for lizeth from 127.0.0.1 … ED25519 SHA256:A36M7//ItYbT61…`. Así se sabe qué persona (qué clave) entró, aunque varias usen la misma cuenta, y se puede revocar justo esa clave.

3. En `sshd_config`, un **`#` no** inicia un comentario a mitad de línea. Si se escribiera `ForceCommand internal-sftp -d /www -u 0002 # solo sftp`, el texto `# solo sftp` pasaría a ser **parte del comando**: `internal-sftp` recibiría argumentos inválidos y la sesión SFTP fallaría. Lo mismo con `Banner`, que buscaría un archivo con ese nombre. Por eso los comentarios van siempre en su propia línea.

Parte C — El equipo de comunicación (SFTP enjaulado)

`bash endurecer_ssh.sh lizeth`:

- creó los grupos `sshadmins` y `sftp-web`;
- creó la cuenta `comunicacion` (`/usr/sbin/nologin`, grupos `sftp-web` y `webadmin`) con su clave ed25519;
- validó con `sshd -t`, reinició ssh e instaló fail2ban.

TODO del script (la jaula):

```
chown root:root /srv /srv/horizonte
chmod 755 /srv /srv/horizonte
```

1. Error y regla. Antes, `/srv/horizonte` era `drwxrwsr-x root:webadmin`, escribible por el grupo, del Lab 2. Para comprobarlo se volvió a dar escritura al grupo un momento (`chmod g+w /srv/horizonte`). `sftp` se cortó (`Connection closed`) y el registro mostró:

```
sshd[2195]: fatal: bad ownership or modes for chroot directory "/srv/horizonte"
```

Regla de sshd: la carpeta del `ChrootDirectory` **y todas sus carpetas superiores** (`/srv`, `/`) deben ser de **root** y **no** escribibles por el grupo ni por otros. Si el usuario enjaulado pudiera escribir en la raíz de su jaula, podría crear o reemplazar archivos que los programas usan como `/etc` o `/lib` y escapar. Con `chmod g-w` volvió a funcionar (`Changed root directory to "/srv/horizonte"`). Lo que sí debe ser escribible es la subcarpeta `www` (`root:webadmin 2775`), donde publica el equipo.

2. Prueba de SFTP:

```
sftp> pwd
Remote working directory: /www
sftp> put /root/lab11/ssh/issue.net aviso.txt
sftp> chmod 664 aviso.txt
sftp> ls -l aviso.txt
-rw-rw-r--    ? 1003     1002          250 Oct 25 11:03 aviso.txt
sftp> cd /etc
stat remote: No such file or directory
```

- Empieza en `/www` (`-d /www`).
- El archivo queda **`comunicacion webadmin 664`**: el grupo lo da el setgid de `www` y los permisos, `-u 0002`.
- `cd /etc` falla porque, para él, `/` es `/srv/horizonte`.
- Muestra `1003 1002` en vez de nombres porque dentro de la jaula no existe `/etc/passwd`.

3. `ssh -i ~/.ssh/comunicacion_ed25519 comunicacion@127.0.0.1 id` → **`This service allows sftp connections only.`**: `ForceCommand internal-sftp` reemplaza cualquier comando y `PermitTTY no` niega la terminal.

Parte D — fail2ban (2 TODO en `fail2ban/horizonte-ssh.local`)

```
journalmatch = _SYSTEMD_UNIT=ssh.service + _COMM=sshd
maxretry = 4
findtime = 10m
bantime = 1h
```

1. `fail2ban-client status sshd` → `Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd`, y `systemctl list-units` muestra la unidad **`ssh.service`**. En Ubuntu 24.04 se llama `ssh.service`, no `sshd.service` como espera el filtro por defecto. Sin el `journalmatch` correcto, fail2ban no vería ningún ataque.

Problema encontrado: los mensajes de sshd **no llegaban al journal**, solo a `/var/log/auth.log`. `/dev/log` era el socket de **`/usr/sbin/syslogd`** (paquete `inetutils-syslogd`), y no un enlace a journald. `/var/log/dpkg.log` mostró que se instaló el **23/10 a las 21:56:54**, como dependencia de `inetutils-telnetd` al activar TELNET en el Lab 10. El script de ese lab purgó `telnetd`, pero dejó `inetutils-telnetd` y `inetutils-syslogd`. Corrección:

```
apt-get purge -y inetutils-syslogd
ln -sf /run/systemd/journal/dev-log /dev/log
systemctl restart ssh fail2ban
```

Purgar `inetutils-syslogd` también quitó el demonio `inetutils-telnetd` que había quedado. Después, el journal registró `Accepted publickey for lizeth … ED25519 SHA256:A36M7//…` y el puerto 23 siguió cerrado.

2. Ataque simulado. Desde un contenedor temporal `tw1-cliente-fail2ban` (IP **172.18.0.3**) en la misma red Docker, con 6 intentos `ssh intruso@172.18.0.2`:

Intento	Respuesta	Qué pasó
1 a 3	`Permission denied (publickey)`	sshd registra `Invalid user intruso from 172.18.0.3`; fail2ban cuenta cada línea (`Found`)
4 a 6	`Connection refused`	fail2ban ya bloqueó la IP: nftables la rechaza antes de llegar a sshd

En modo `aggressive` cada conexión suma más de una falla (`Invalid user` y `Connection closed … [preauth]`), por eso se alcanzó `maxretry = 4` en el tercer intento y en menos de 2 segundos.

- `fail2ban-client status sshd` → `Total failed: 20`, `Total banned: 2`, `Banned IP list: 172.18.0.3`.
- `nft list set inet f2b-table addr-set-sshd` → `elements = { 172.18.0.3 }`, con la regla `tcp dport 22 ip saddr @addr-set-sshd reject with icmp port-unreachable`.
- `/var/log/fail2ban.log`:

```
2026-10-25 11:30:25,720 fail2ban.actions [2413]: NOTICE [sshd] Ban 172.18.0.3
2026-10-25 11:46:10,962 fail2ban.actions [2413]: NOTICE [sshd] Unban 172.18.0.3
2026-10-25 11:47:03,343 fail2ban.actions [2413]: NOTICE [sshd] Ban 172.18.0.3
```

Las líneas `observer … # -> 2.0` corresponden a `bantime.increment`: si la IP reincide, el siguiente bloqueo dura el doble.

3. `fail2ban-client set sshd unbanip 172.18.0.3` → `1`, y el conjunto de nftables quedó vacío.

Parte E — Túneles

1. Desde **mi PC con Windows** (OpenSSH_for_Windows_9.5p2), con una clave propia `lizeth-pc-2026` protegida por frase:

- La clave se creó en la PC; solo la `.pub` se copió a `authorized_keys` de lizeth, y la privada no salió de Windows.
- Primero apareció **`WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!`**: la PC tenía guardada la clave vieja del servidor (la de la imagen), anterior a regenerarla en el Lab 10. Se verificó por otro canal con `docker exec … ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub` → `SHA256:KueRdm7E…`, igual a la del aviso. Solo entonces se borró esa entrada con `ssh-keygen -R "[localhost]:2222"` y se aceptó la huella verificada.
- Las primeras conexiones se cortaron (`Connection aborted` / `Connection closed`) porque la pregunta de la huella quedó abierta más de **20 s** y `LoginGraceTime 20` las cerró. Con la huella ya guardada, conectó sin problema.

```
ssh -N -L 9081:127.0.0.1:8081 -p 2222 -i $HOME\.ssh\id_ed25519_horizonte lizeth@127.0.0.1
curl.exe http://127.0.0.1:9081/eco   →  {"metodo": "GET", "ruta": "/eco", … "cliente":
"127.0.0.1" …}
```

2. Camino del paquete:

```
PC: curl ─► 127.0.0.1:9081 (ssh escucha en la PC)
            ╟ túnel cifrado (chacha20-poly1305) por el puerto 2222 → 22 del contenedor
            ▼
srv-horizonte: sshd ─► 127.0.0.1:8081 (API eco, solo local) ─► respuesta por el mismo túnel
```

La API ve la petición como `"cliente": "127.0.0.1"` porque quien la hace es sshd dentro del servidor. La API sigue cerrada al mundo, y solo un administrador con clave puede usarla.

- **`-D 1080` (SOCKS):** túnel **dinámico**. ssh abre un proxy SOCKS en el puerto 1080 de la PC y cada conexión que pasa por él sale desde el servidor hacia cualquier destino. Sirve, por ejemplo, para navegar como si se estuviera en la red del colegio.

- `-J` **(salto):** se conecta a un servidor interno **pasando por** un servidor de salto (*bastion*), por ejemplo `ssh -J lizeth@bastion lizeth@srv-interno`. Solo el bastión se expone a internet y la sesión va cifrada de extremo a extremo.

Parte F — Verificación

`bash verificar_lab11.sh lizeth` → **Resultado: 15 de 15 comprobaciones.**

Reflexión

¿Por qué endurecer SSH con claves y AllowGroups es más importante que cambiar el puerto 22? Cambiar el puerto es "seguridad por oscuridad": reduce el ruido de los bots más simples, pero un escaneo de puertos encuentra el nuevo en segundos y el servidor sigue aceptando contraseñas adivinables. Las claves ed25519 eliminan el riesgo de raíz: no hay contraseña que adivinar ni que robar en la red. `AllowGroups` reduce quién puede siquiera intentarlo. `PermitRootLogin no` obliga a entrar con una cuenta personal (trazable con `LogLevel VERBOSE`). fail2ban corta a quien insiste. Cambiar el puerto puede sumarse como capa extra, pero nunca reemplaza estas medidas.

Fuentes:

- Security Magazine. (2021, 26 de abril). *Unsecured computer attacked 51 times per minute*. https://www.securitymagazine.com/articles/95082-unsecured-computer-attacked-51-times-per-minute

Anexo B — BITACORA_IA.md

Bitácora de uso de IA — Laboratorio 11 (SSH endurecido, SFTP, fail2ban y túneles).

Estudiante: Lizeth Rodríguez · Entorno: contenedor `horizonte-servidor-lab02` (`srv-horizonte`, Ubuntu 24.04, OpenSSH 9.6p1, fail2ban + nftables), PC con Windows (OpenSSH_for_Windows 9.5p2) · Fecha: 25/10/2026

Reglas que respeté:

- **Dos sesiones** abiertas en el servidor antes de tocar sshd (ventana de trabajo y salvavidas). `sshd -t` validó la configuración antes de cada reinicio.
- **Ninguna clave privada** se pegó en la IA ni aparece en las capturas. La clave de mi PC (`lizeth-pc-2026`) se creó en Windows con frase; solo la `.pub` se copió al servidor.
- **No acepté huellas sin verificar**: ante "REMOTE HOST IDENTIFICATION HAS CHANGED" comparé la huella por otro canal (`docker exec … ssh-keygen -lf`) antes de borrar solo esa entrada.
- El ataque de fuerza bruta se hizo **solo contra mi servidor del curso**, desde un contenedor temporal en la misma red Docker, con 6 intentos de un usuario inexistente. Al final desbloqueé la IP.

#	Qué pregunté	Qué respondió la IA	Cómo lo verifiqué	Qué aprendí
1	Cómo completar los 6 TODO de `10-horizonte.conf` sin romper el acceso.	Cambiar `PermitRootLogin`/`PasswordAuthentication` a `no`, `AllowGroups sshadmins sftp-web` y el bloque `Match Group sftp-web` al final; comentarios siempre en su propia línea.	`grep` del archivo, `sshd -t` en el script y `sshd -T` con los valores nuevos; root y contraseñas rechazados, lizeth entra.	`Match` va al final y los comentarios al final de línea rompen `ForceCommand`.
2	Por qué `sshd_config.d/` y "gana la primera aparición".	El `Include` está al inicio de `sshd_config`; en sshd vale el primer valor leído.	`sshd -T` mostró nuestros valores aunque el `sshd_config` de Ubuntu no se tocó.	Configurar sin editar el archivo del paquete.
3	Qué hacer con `/srv/horizonte` para la jaula.	Debe ser `root:root` y no escribible por el grupo, igual que sus superiores; `www` sigue `root:webadmin 2775`.	Antes: `drwxrwsr-x root:webadmin`. Con `chmod g+w` de prueba, sftp se cortó y `auth.log` mostró `bad ownership or modes for chroot directory`; con `g-w` volvió a `/www`.	Por qué sshd exige la raíz de la jaula solo de root.
4	Por qué `journalctl` no mostraba nada de `comunicacion`.	Comparar el journal con `/var/log/auth.log` y ver quién escucha en `/dev/log` (`ss -xlp`, `ps`).	`/dev/log` era el socket de `/usr/sbin/syslogd`; `dpkg -S` → `inetutils-syslogd`; `dpkg.log` → instalado el 23/10 21:56:54 junto con telnetd (Lab 10).	Una dependencia que queda instalada puede romper otro servicio días después.
5	Cómo arreglarlo sin romper otra cosa.	Simular primero (`apt-get -s purge`), purgar `inetutils-syslogd`, restaurar el enlace `/dev/log` y reiniciar ssh y fail2ban.	La simulación mostró que también se iba `inetutils-telnetd`; después el journal registró `Accepted publickey … SHA256:A36M7//…` y el puerto 23 siguió cerrado.	Simular antes de borrar paquetes.

#	Qué pregunté	Qué respondió la IA	Cómo lo verifiqué	Qué aprendí
6	El nombre de la unidad para `journalmatch`.	En Ubuntu 24.04 es `ssh.service`; el filtro por defecto busca `sshd.service`.	`systemctl list-units` → `ssh.service`; `Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd`.	Comprobar el nombre real de la unidad.
7	Cómo simular el ataque desde otro equipo.	Un contenedor temporal en `alternativa_docker_default` que intente 6 veces `ssh intruso@172.18.0.2`; desde el propio servidor no sirve por `ignoreip 127.0.0.1/8`.	Alpine falló (comillas de PowerShell y paquete no encontrado); con la imagen `horizonte-servidor:24.04`: 3× `Permission denied` y 3× `Connection refused`, `Ban 172.18.0.3` en el log y la IP en `addr-set-sshd`.	PowerShell 5.1 borra las comillas dobles internas al pasar argumentos.
8	Por qué se bloqueó al tercer intento si `maxretry = 4`.	En modo `aggressive` cada conexión genera varias líneas que cuentan (`Invalid user` y `Connection closed … [preauth]`).	`fail2ban.log` muestra varios `Found` por segundo antes del `Ban`; `Total failed: 20` entre los dos ataques.	Cómo cuenta fail2ban las fallas.
9	Qué hacer con "REMOTE HOST IDENTIFICATION HAS CHANGED" desde Windows.	Es la clave vieja (de antes del Lab 10). Verificar la huella por otro canal y solo entonces `ssh-keygen -R "[localhost]:2222"`.	La huella del aviso y la de `ssh-keygen -lf` en el servidor coincidieron: `SHA256:KueRdm7E…`.	Aplicar en la práctica la reflexión del Lab 10.
10	Por qué la conexión desde Windows se cortaba (`Connection aborted`).	`LoginGraceTime 20`: la pregunta de la huella quedó abierta más de 20 s y el servidor cerró.	`ssh -v` llegaba a `NEWKEYS` y se cortaba; el journal mostraba la conexión de 172.18.0.1 sin intento de clave. Con la huella guardada, conectó.	Mis propias medidas de seguridad también tienen efectos que hay que entender.
11	Cómo explicar el túnel local, `-D` y `-J`.	`-L 9081:127.0.0.1:8081` publica en la PC un puerto que sale desde el servidor; `-D` crea un proxy SOCKS dinámico; `-J` salta por un bastión.	`curl.exe http://127.0.0.1:9081/eco` devolvió el JSON con `"cliente": "127.0.0.1"`; el verificador → ✅ túnel.	Alcanzar servicios internos sin exponerlos.
12	Cuántos intentos recibe un SSH en internet.	Los estudios con honeypots registran miles por día.	Lo contrasté con Security Magazine (2021): ~73 000 ataques en 24 h, 57 763 de fuerza bruta SSH.	Citar una fuente, no un número de la IA.

Conclusión

La IA me ayudó sobre todo a diagnosticar. El problema más importante no estaba en la guía: un `syslogd` instalado como dependencia de TELNET en el Lab 10 le quitaba los registros al journal, y sin ellos fail2ban habría quedado ciego. Lo confirmé con `ss`, `dpkg -S` y `dpkg.log` antes de cambiar nada. Cada respuesta la verifiqué con `sshd -T`, el registro, `nft` o el verificador. El servidor quedó solo con claves, sin root, con SFTP enjaulado para comunicación, fail2ban bloqueando en nftables y la API interna accesible solo por túnel. Resultado: **15 de 15**.